

NoSQL Databases

Introduction

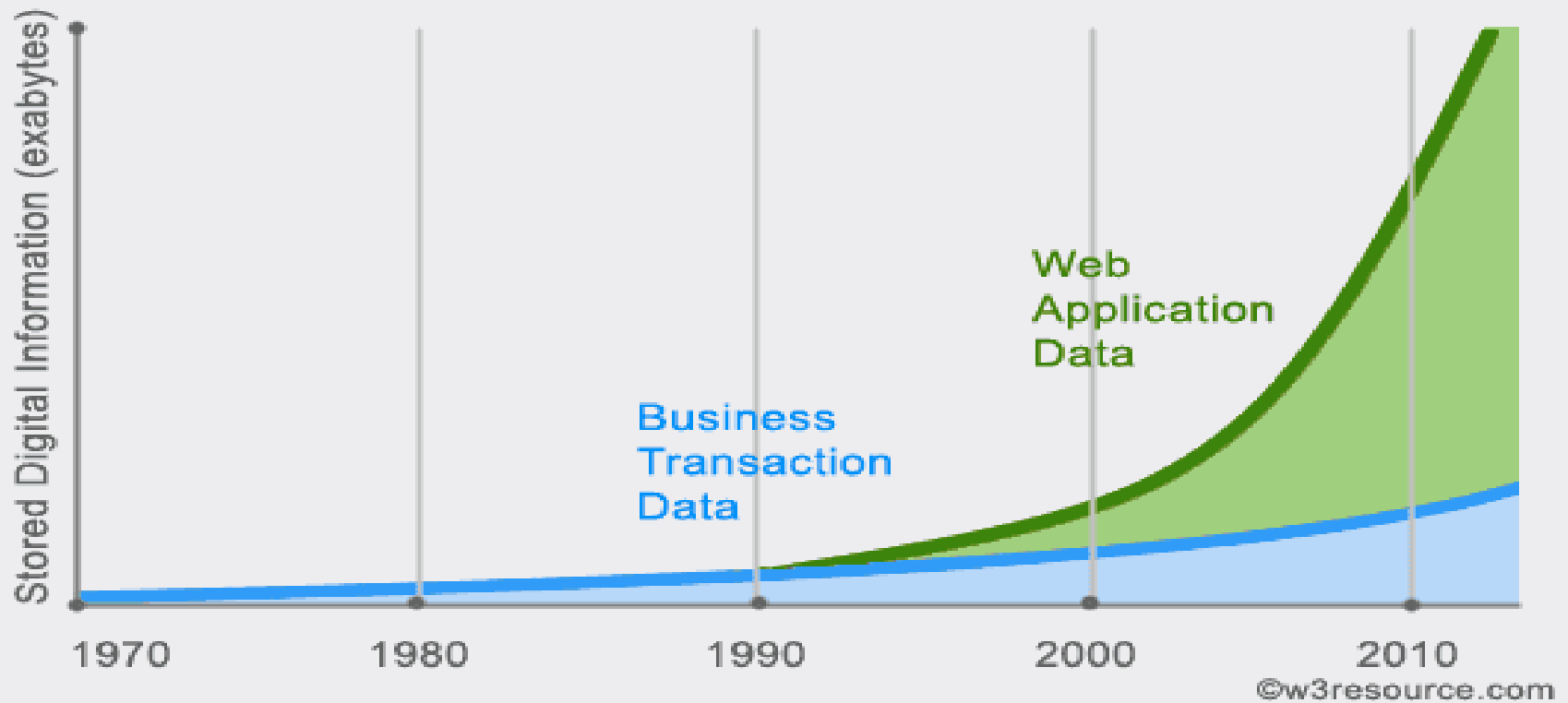
Agenda

- Current trends in data management & computing
- Big Data
- What is NoSQL ?
- Relational vs. NoSQL databases
- NoSQL Pros and Cons
- Data model of aggregates
- Models of data distribution
 - Scalability
 - Distribution models
- Types of NoSQL databases

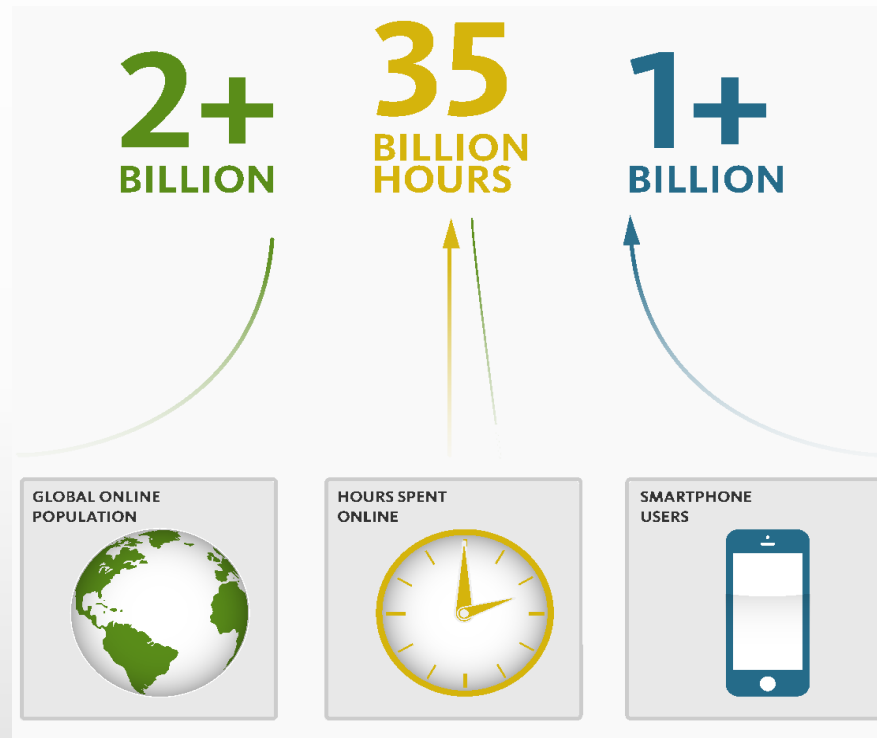
Current Trends in Data Management & Computing

Data Growth

Web Applications Driving Data Growth



Big Users



- It is common to start a Web-based system and have millions of users within a few months

Cloud Computing



- Everything is in Cloud
 - flexibility and distributed nature of the systems

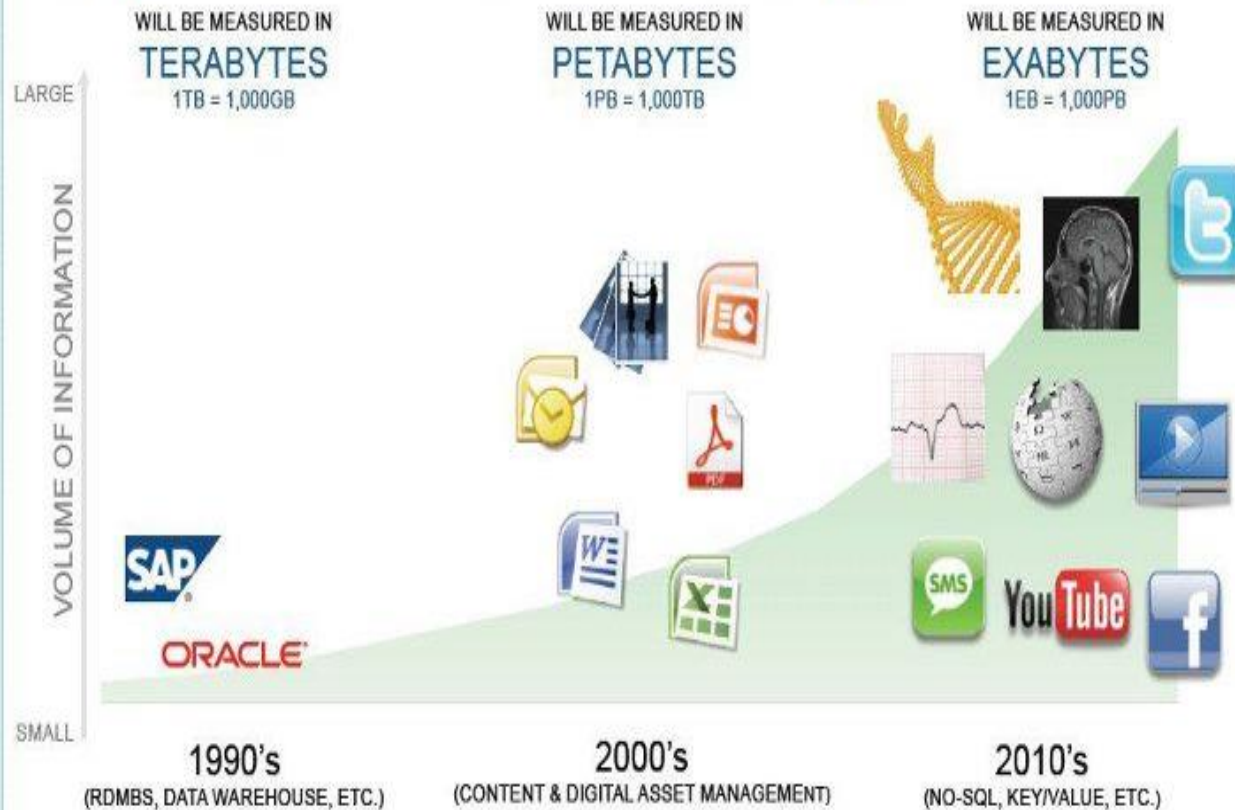
Big Data

Introduction

“Big data is high volume, high velocity, and/or high variety information assets that require new forms of processing to enable enhanced decision making, insight discovery and process optimization.” (Gartner, 2012)



Big Data Will Scale To Exabytes



Sources of Big Data

- Social networks
 - this data is huge, but the volumes are relatively limited
- Logs of various web/email servers or routers
 - growing beyond limits
- Sensor networks
 - this sector is expected to grow even faster
- Internet of things (IoT)
- Computer-driven machines, like airplanes:
 - one overseas flight of Boeing generates 640 TB of data

RDBMS for Big Data

- Relational **schema**
 - data in tuples
 - **a priori** known schema
- Schema **normalization**
 - data split into tables (3NF)
 - queries merge the data
- **Transaction** support
 - trans. management with ACID
 - Atomicity, Consistency, Isolation, Durability
 - safety first
- But current data are naturally **flexible**
- **Inefficient** for large data
- Slow in **distributed** environment
- **Full transactions** very inefficient in **distributed** environment

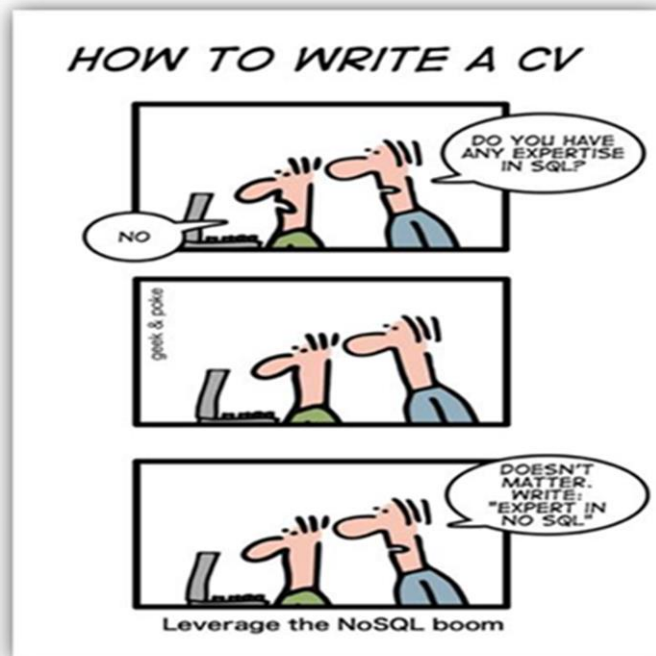
What is NoSQL ?

NoSQL Databases

“NoSQL is an accidental term with no precise definition”

- **first used** at an informal meetup in **2009** in San Francisco (presentations from Voldemort, Cassandra, Dynomite, HBase, Hypertable, CouchDB, and MongoDB)

[Sadalage & Fowler: NoSQL Distilled, 2012]



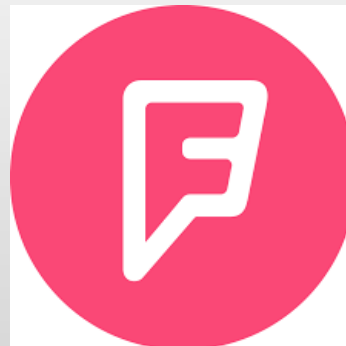
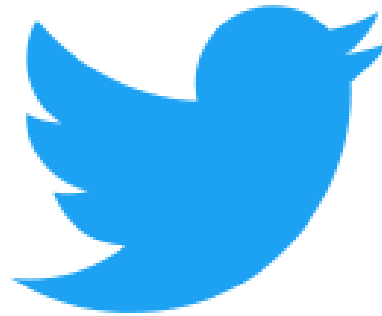
NoSQL Databases

- NoSQL: Database technologies that are (mostly):
 - **Not using** the **relational** model (nor the SQL language)
 - Designed to run on **large clusters** (horizontally scalable)
 - **No schema** - fields can be freely added to any record
 - Open source
 - Based on the needs of 21st century web estates

[Sadalage & Fowler: NoSQL Distilled, 2012]

- Other characteristics (often true):
 - easy **replication** support (fault-tolerance, query efficiency)
 - **simple** API
 - **eventually** consistent (not ACID)

Who uses NoSQL ?

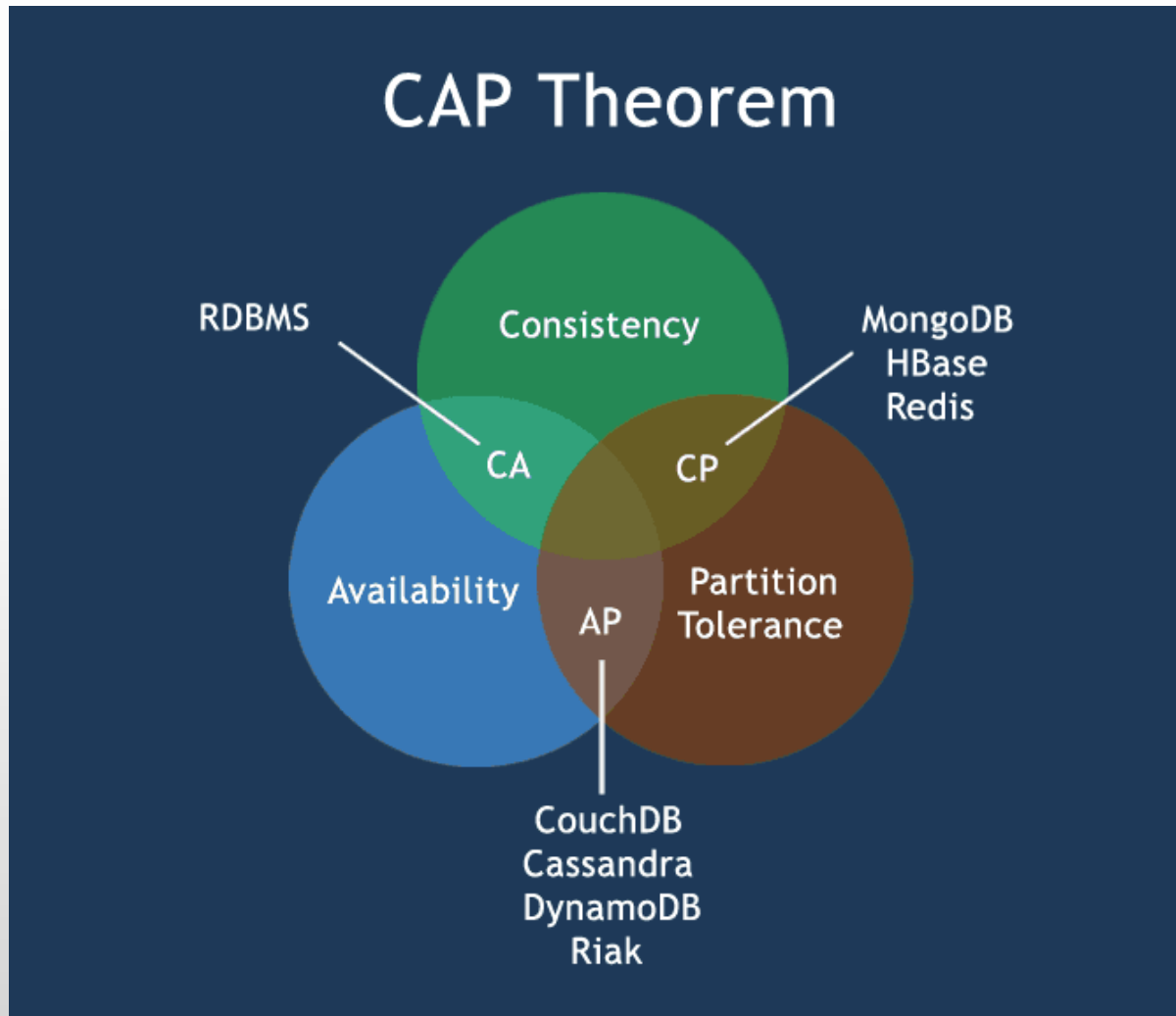


RDBMS vs NoSQL

RDBMS	NoSQL
Structured and organized data	Unstructured or semi structured data
Structured Query Language (SQL)	No declarative query language
Schema - centric	Schema - free
Data and its relationships are stored in separate tables	Distributed storage
Complex data relationships	Very simple relationships
Joins	Avoid joins
Database-centric	Application-centric or developer-centric
Well defined standards	Standards not yet evolved

CAP Theorem

CAP Theorem



CAP Theorem

- **Consistency** : data in the database remains consistent after the execution of an operation

e.g : after an update operation, all clients see the same data

- **Availability** : system is always on
- **Partition Tolerance** : system continues to function even the communication among the servers is unreliable

BASE Concept

BASE

BASE is vague term often used as contrast to ACID

- **Basically Available**

The system works basically all the time

Partial features can occur, but without total system failure

- **Soft state**

The system is in flux, non-deterministic state

Changes occur all the time

- **Eventual consistency**

The system will be in some consistent state

At some time in future

Data model of Aggregates

Data model

- The model by which the database organizes data
- Each NoSQL database type has a different data model.

Key-value, document, column-family, graph

The first three are oriented on aggregates

Aggregates

An aggregate

- A data unit with a complex structure
- A collection of related objects treated as a unit
 - unit for data manipulation and management of consistency
- Relational model is aggregate-ignorant

Aggregates

An aggregate

- Data aggregates are a core concept in many NoSQL databases, representing a collection of related data treated as a single unit for storage, manipulation, and consistency.
- Unlike relational databases, where data is spread across multiple tables with relationships enforced by foreign keys, NoSQL databases, particularly those oriented toward aggregates, organize data into self-contained units that make it easier to scale and manage.

Aggregates

Data Model of Aggregates

1. The data model refers to the way a database organizes and stores data.
 - NoSQL databases use different data models depending on the type:
 - Key-Value Stores: Store data as simple key-value pairs (e.g., Redis, DynamoDB).
 - Document Stores: Store data as documents (often JSON or BSON) (e.g., MongoDB).
 - Column-Family Stores: Store data in columns grouped into families (e.g., Cassandra).
 - Graph Databases: Store data in the form of nodes and edges to represent relationships (e.g., Neo4j).
 - The first three types—Key-Value, Document, and Column-Family stores—are often referred to as aggregate-oriented databases because they organize data into aggregates.

Aggregates

- Aggregates in NoSQL databases refer to units of related data treated as a whole.
- They help with data locality, performance, and scalability, especially in distributed systems. Key-value stores, document stores, and column-family stores are aggregate-oriented because they are designed to work with these grouped data units.
- Aggregates reduce the need for complex joins and improve concurrency and consistency control by focusing on smaller, manageable data units.

NoSQL Databases : Aggregate-oriented

- Many NoSQL stores are aggregate-oriented
 - Aggregates give the database information about which bits of data will be manipulated together
 - Minimize the number of nodes accessed during a search
 - Impact on concurrency control

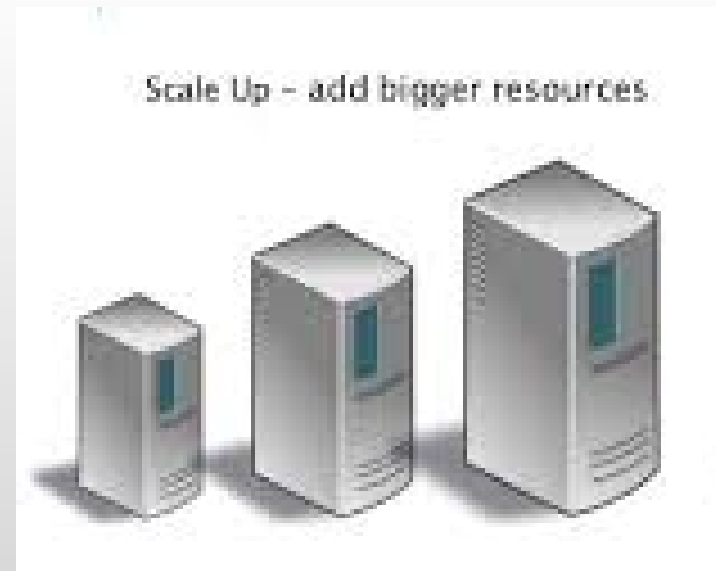
Models of Data Distribution

Scalability of Database systems

- **Scalability** => handling growing amounts of data and queries without losing performance
- Two general approaches :
 - vertical scalability
 - horizontal scalability

Vertical Scalability (Scaling up)

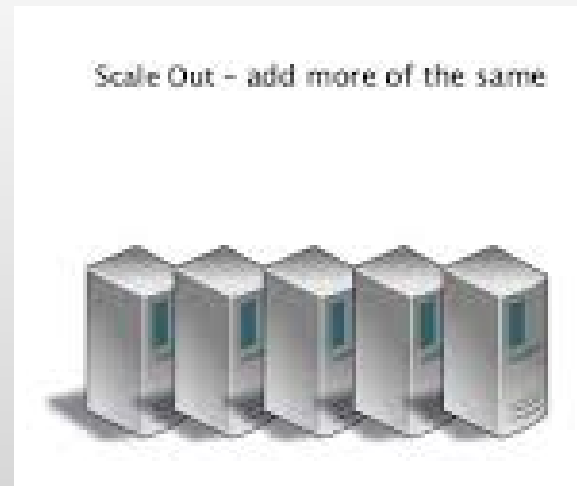
- Involve larger and more powerful machines
- Traditional choice
- Works in many cases but it has some drawbacks
 - Higher costs
 - Data growth limit
 - Proactive provisioning
 - Vendor lock-in



Horizontal Scalability (Scaling out)

System is distributed across multiple machines/nodes.

- Commodity machines, cost effective
- Provides higher scalability than vertical approach
- Introduces new problems like synchronization, consistency, partial failure handling, etc .



Distribution models

- Two generic ways of data distribution
 - Replication => same data is copied over multiple nodes

Master-slave or peer-to-peer

- Sharding => different data chunks are put on different nodes

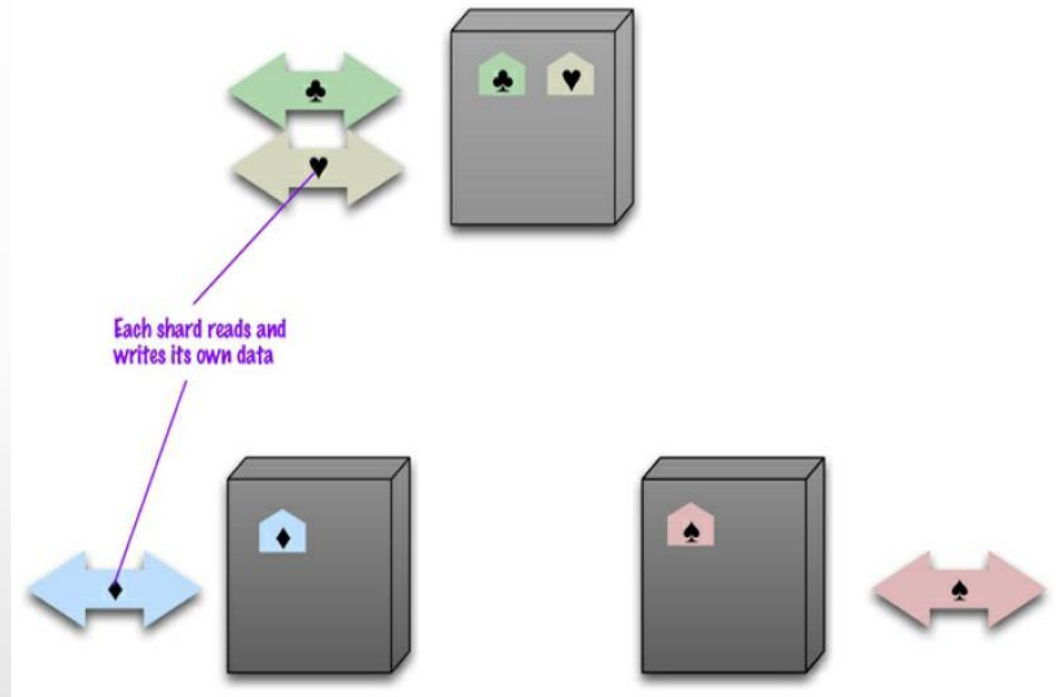
(data partitioning)

Distribution model : Single Server

- Running the database on a single machine is always the preferred scenario
- It can make sense to use a NoSQL database on a single server
 - other advantages : flexible data model, simplicity
 - Graph databases : If the graph is “almost” complete, it is difficult to distribute it

Sharding (Data Partitioning)

- Placing different parts of the data onto different servers
- Different people are accessing different parts of the datasets

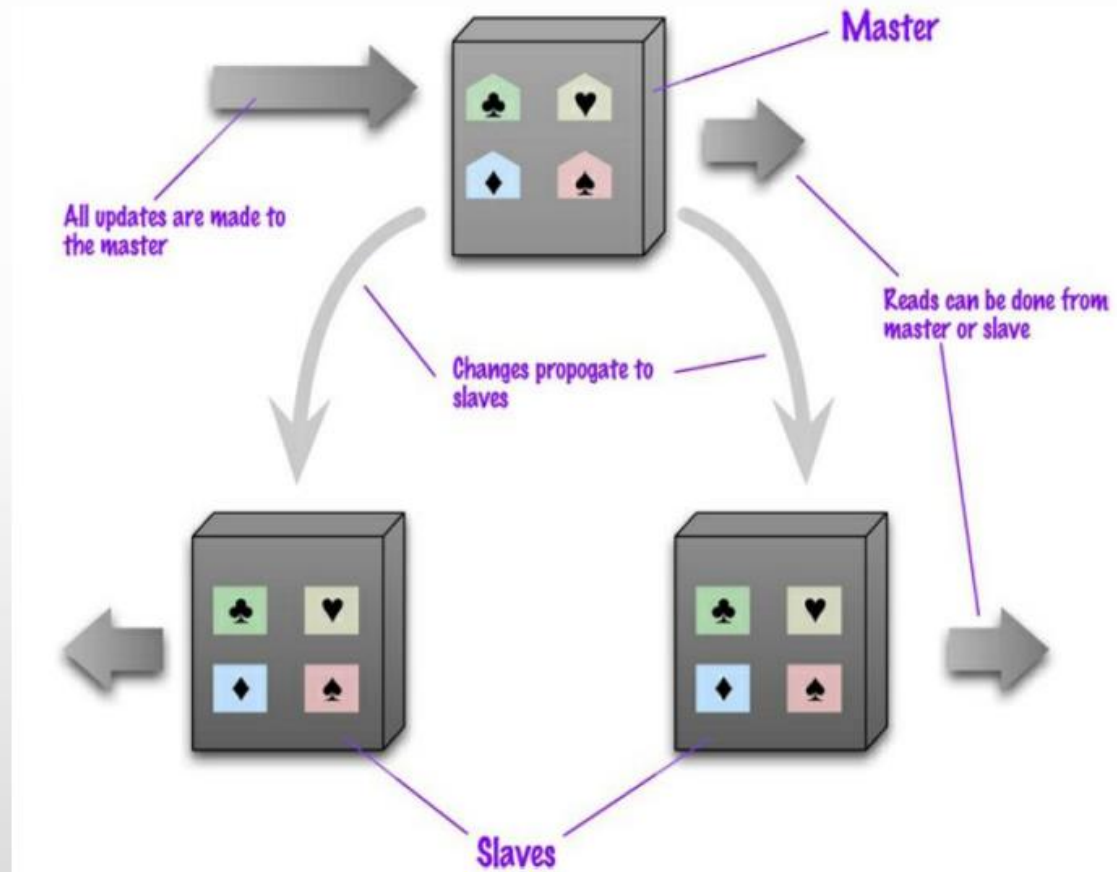


Distribution model : Sharding

- Data accessed together is kept together
- Arrange the data on the nodes
 - Based on the physical location
 - Keep the load balanced
- Many NoSQL databases offer auto-sharding
- A node failure makes shard's data unavailable

Master – slave Replication

- Replicate data across multiple nodes
- One node is designated as primary (master), others as secondary (slaves)
- Master is responsible for processing all updates to the data
- Reads from any node

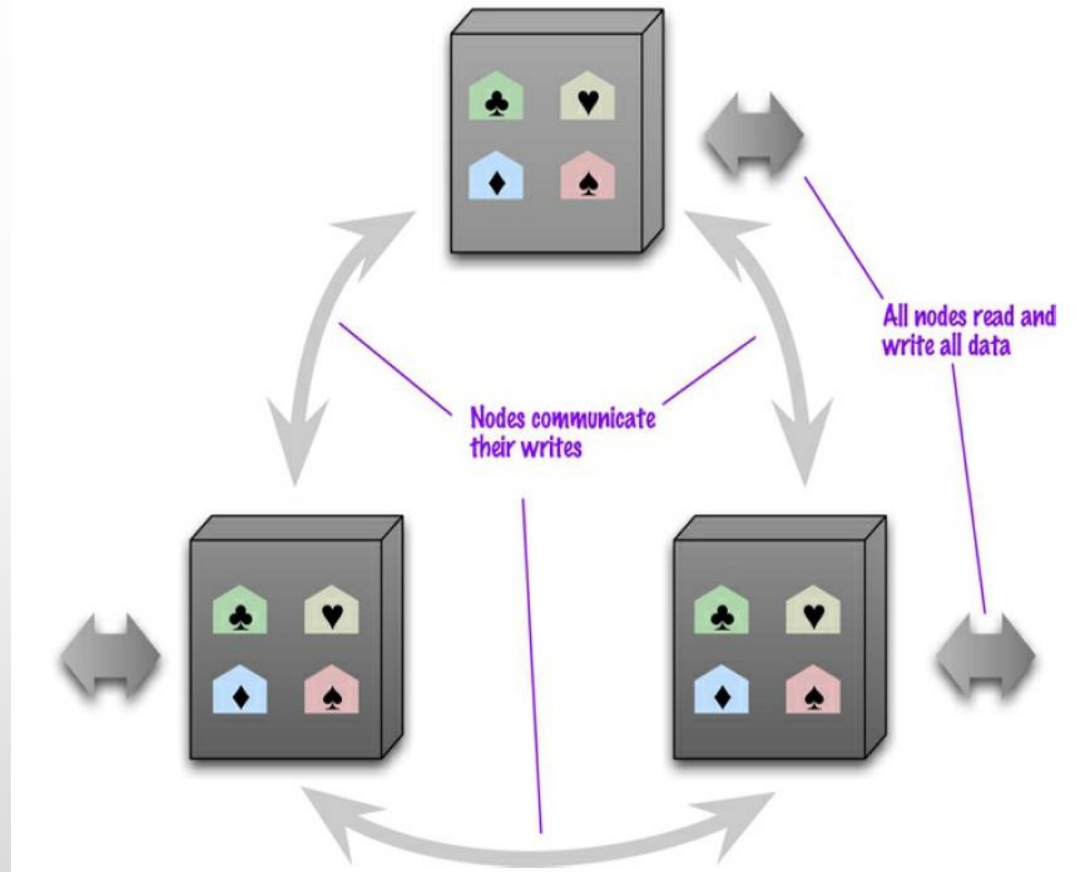


Master – slave Replication

- For scaling a read-intensive application
 - More read requests -> more slave nodes
 - If the master fails -> the slave can still handle read requests
 - A slave can become a new master quickly
- Limited by ability of the master to process updates
- Masters are selected manually or automatically

Peer-to-peer Replication

- No master, all the replicas are equal
- Every node can handle a write and then spreads the update to the others



Peer-to-peer Replication

- Problem : Consistency
 - Users can write simultaneously at two different nodes
- Solution :
 - When writing, the replicas coordinate to avoid conflict

Types of NoSQL Databases

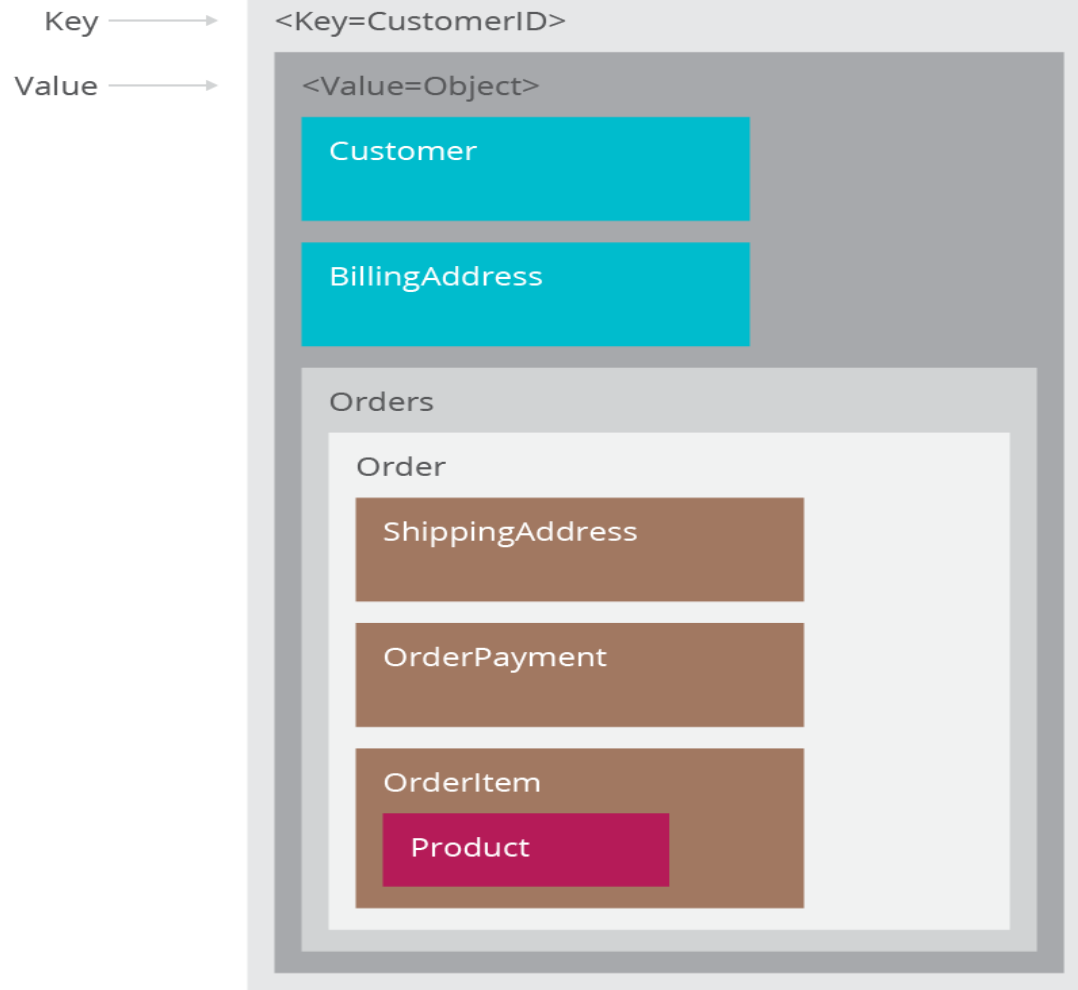
Key - Value Stores: Representatives



Key - Value Stores

- Key-value stores are the simplest NoSQL data stores to use from an API perspective.
- Designed to handle huge amount of data.
- Based on Amazon's Dynamo paper.
- Allows developer to store schema-less data.
- Database stores data as hash table where each key is unique and the value can be string, JSON, BLOB etc.
- A key may be strings, hashes , lists, sets, sorted sets.
- Key-value stores can be used as collections, dictionaries, associative arrays etc.

Key - Value Stores



Document Databases: Representatives



MS Azure
DocumentDB

Document Databases

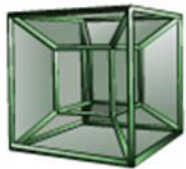
- A collection of documents.
- Data in this model is stored inside documents.
- A document is a key value collection where the key allows access to its value.
- Documents are not typically forced to have a schema and therefore are flexible and easy to change.
- Documents are stored into collections in order to group different kinds of data.
- Documents can contain many different key value pairs, or key array pairs, or even nested documents.

Document Databases

<Key=CustomerID>

```
{
  "customerid": "fc986e48ca6" ← Key
  "customer":
  {
    "firstname": "Pramod",
    "lastname": "Sadalage",
    "company": "ThoughtWorks",
    "likes": [ "Biking", "Photography" ]
  }
  "billingaddress":
  { "state": "AK",
    "city": "DILLINGHAM",
    "type": "R"
  }
}
```

Column-family Stores: Representatives



HYPERTABLE



Column-family Stores

- Primarily work on columns and every column is treated individually.
- Values of a single column are stored contiguously.
- Column stores data in column specific files.
- In column stores, query processors work on column too.
- All data within each column data file have the same type which makes it ideal for compression.
- Column stores can improve the performance of queries as it can access specific column data.

Column-family Stores

Column Family

Row

Row KeyX

Column1

name1:value1

Column2

name2:value2

ColumnN

nameN:valueN

Row

Row KeyY

Column1

name1:value1

Column9

name9:value9

ColumnN

nameN:valueN

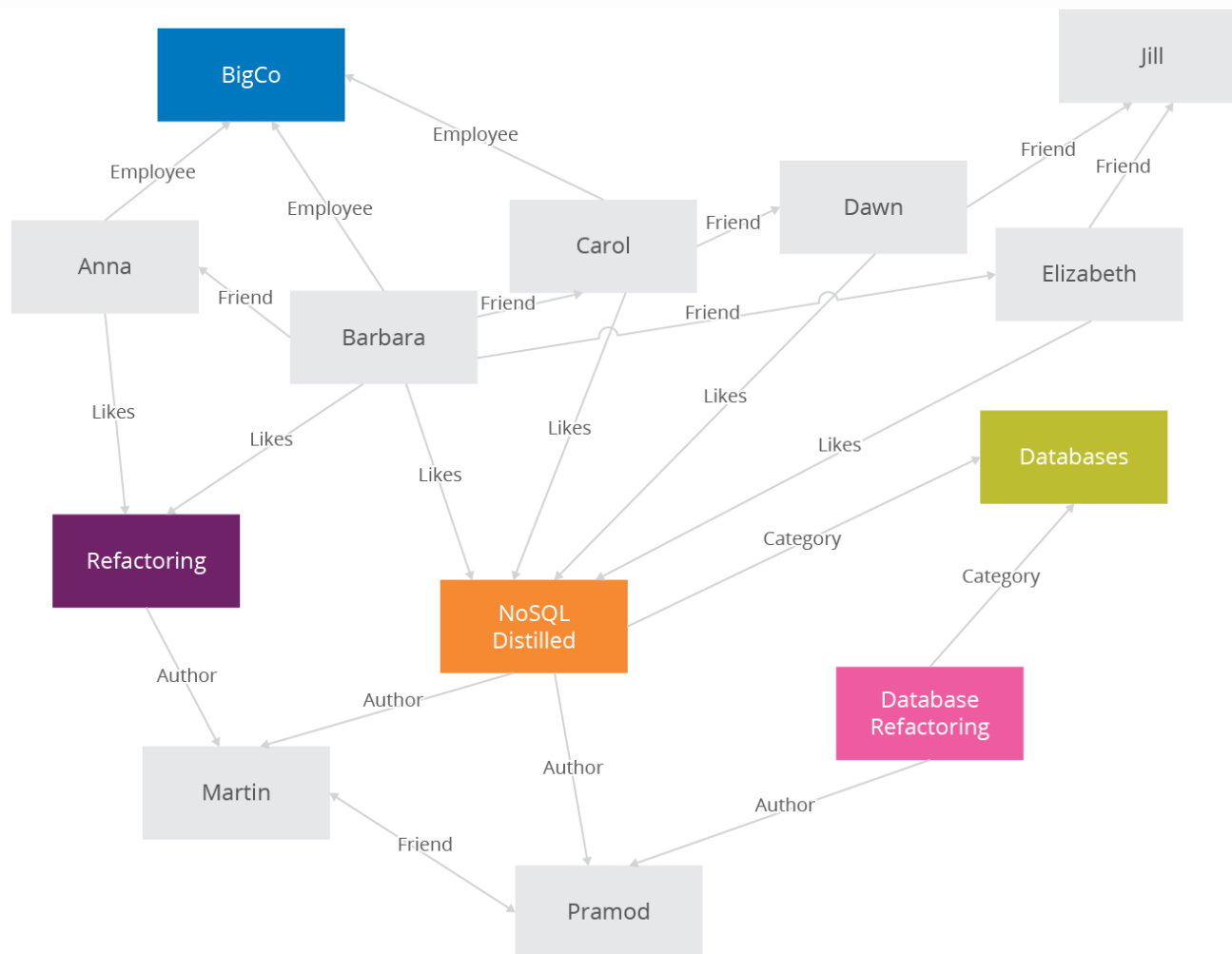
Graph Databases: Representatives



Graph Databases

- Stores data in nodes.
- Capable of elegantly representing any kind of data in a highly accessible way.
- Is a collection of edges and nodes.
- Each node represents as entity, and each edge represents a connection or relationship between two nodes.
- Every node and edge is defined by a unique identifier.
- Each node knows its adjacent nodes.
- As the number of nodes increases, the cost of a local step (or hop) remains the same.

Graph Databases



Why choose NoSQL database ?

Reasons to choose NoSQL databases

- To improve programmer productivity by using a database that better matches an application's needs.
- To improve data access performance via some combination of handling larger data volumes, reducing latency and improving throughput.

Choosing NoSQL databases

Choosing NoSQL Databases

It depends on the system requirements and here are some general guidelines.

- Key-value databases are generally useful for storing session information, user profiles, preferences, shopping cart data.
- Document databases are generally useful for content management systems, blogging platforms, web analytics, real-time analytics, ecommerce-applications.
- Column family databases are generally useful for content management systems, blogging platforms, maintaining counters, expiring usage, heavy write volume.
- Graph databases are very well suited to problem spaces where we have connected data such as social networks, special data, routing information for goods and money, recommendation engines.

References

- Sadalage, P. J., & Fowler, M. (2012). NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence. Addison-Wesley Professional.
- Pramod Sadalage, NoSQL Databases : An Overview, <http://www.thoughtworks.com/insights/blog/nosql--databases--overview>.



Thank you !!!